



Team Name: [to fill]

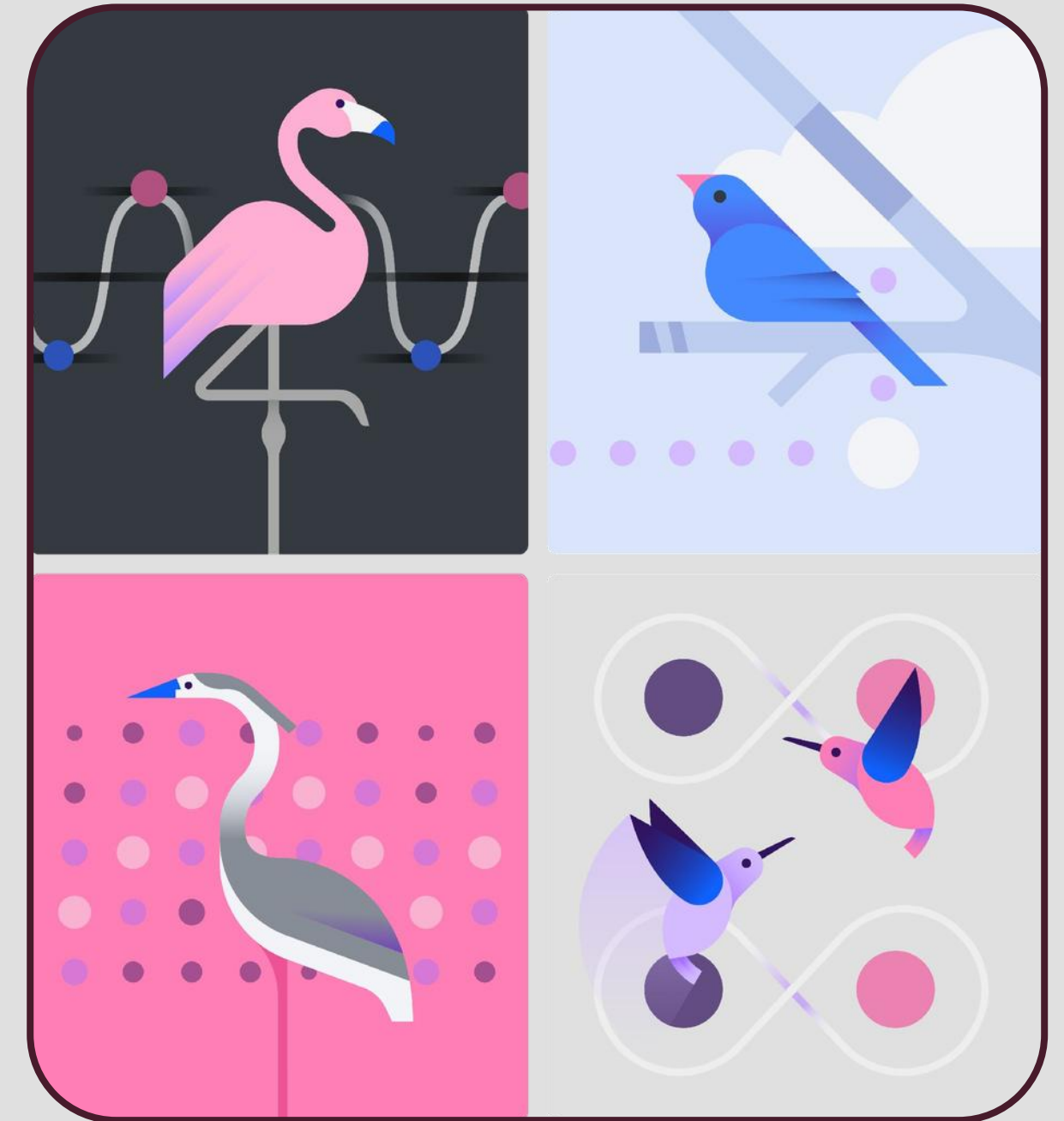
Team Lead Name: [to fill]

Track: [to fill]

Problem Statement title:

Q-SHIELD: continuous device trust with post-quantum-signed evidence

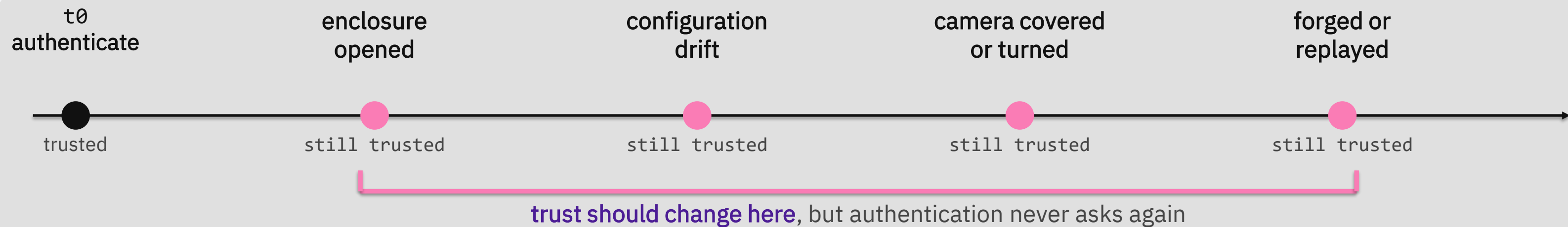
Authentication answers **who are you** once. Q-SHIELD keeps asking **can you still be trusted**.





Problem

A device that authenticated can still be compromised, and nothing asks again



Context

AIoT devices at the edge (sensors, cameras, controllers) authenticate once with a provisioned key, then run unattended.

The gap

Authentication proves who is talking, not whether the device is still in a trustworthy state.

The quantum angle

Evidence signed today with RSA or ECDSA could be forged once large quantum computers exist.





Why It Matters

Who is exposed, and what each attack looks like with and without Q-SHIELD

Who faces it

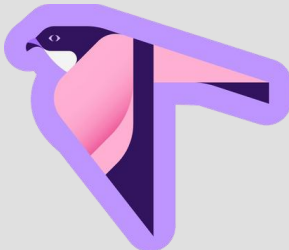
- Research labs and server rooms
- Smart buildings and campuses
- Critical-infrastructure sites
- Any fleet of unattended AIoT devices

Why now

NIST published FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA) in August 2024: evidence signed today should already be post-quantum.

Threat	Authentication alone	With Q-SHIELD
Forged or replayed message	rejected, then forgotten	bounded pressure (≤ 25 points); 3 replays in 10 min hold the score at 65
Enclosure opened	nothing to see	physical factor CRITICAL; score held at 55
Camera covered, frozen or turned	nothing to see	signed camera-health report; visual factor HIGH
Tamper + camera or visual evidence	nothing to see	confirmed incident: score 30, quarantine
Quantum adversary recording traffic	classical signatures forgeable later	ML-DSA-65 signs the evidence; ML-KEM-768 keys the session

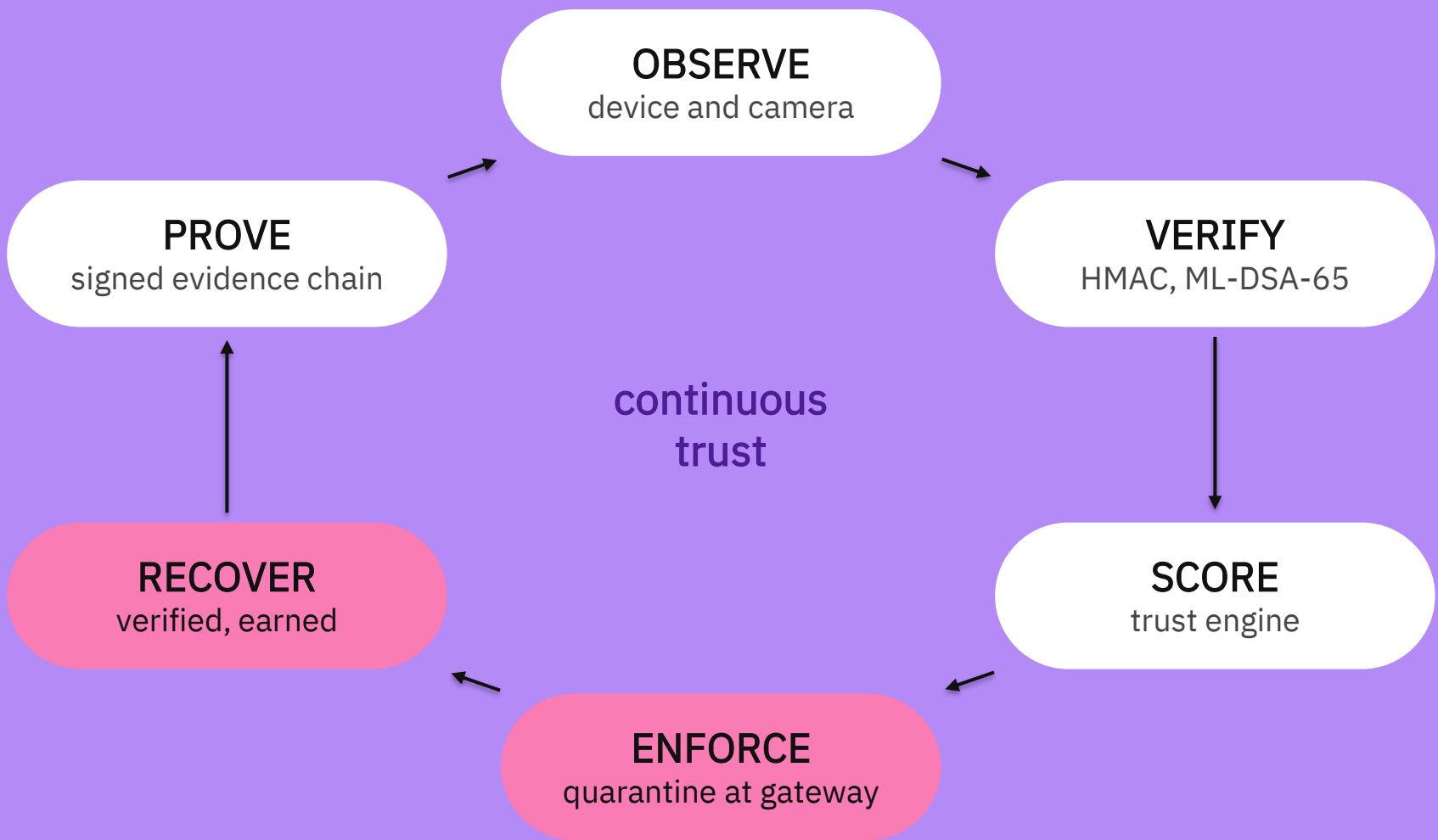
No single signal quarantines a device. Two independent authenticated signals within 60 s do.





Proposed Solution

A closed loop around every device: trust is computed continuously, enforced, and earned back



What we are building

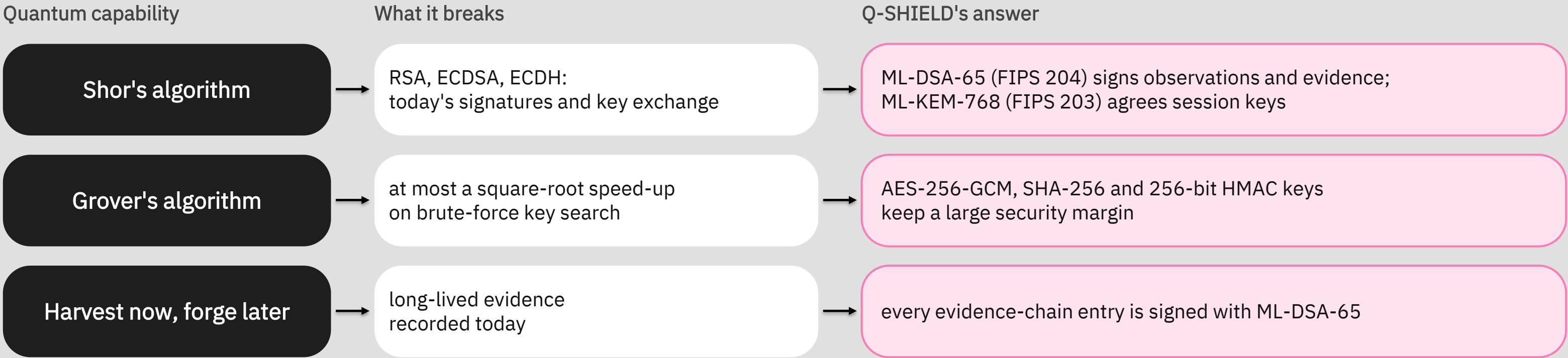
- Continuous six-factor trust engine
every score change lists the reasons that sum to it
- The camera as part of the security boundary
covered, frozen or turned is signed evidence
- Quarantine enforced at the gateway
the device's normal channel is refused (403)
- Verified, earned recovery
health checks first, then trust rebuilt from fresh evidence
- Post-quantum evidence
ML-DSA-65 signatures, ML-KEM-768 sessions





Why Quantum?

Quantum computing is the threat model; post-quantum cryptography is how Q-SHIELD answers it



The cost of going post-quantum (measured, laptop CPU)

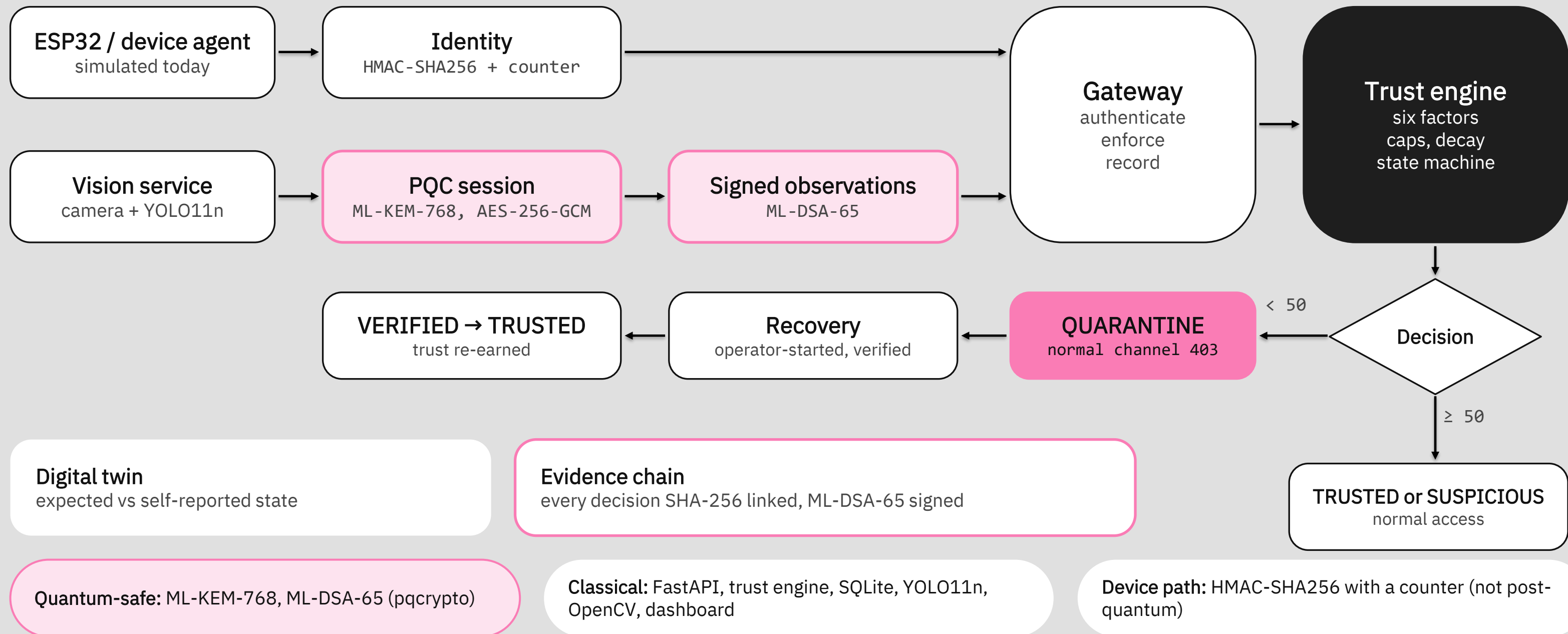
signature size	ML-DSA-65 3309 B vs ECDSA P-256 64 B
sign / verify	9.42 ms / 0.188 ms (median)
ML-KEM-768 encapsulate	0.069 ms (median)
full session handshake	10.0 ms (median)

Honest scope: Q-SHIELD runs no quantum circuits and no quantum hardware. Its quantum-relevant part is post-quantum cryptography on the evidence path; the ESP32 device path is HMAC-SHA256, not a post-quantum signature.



Implementation and Architecture

1 of 3: how the parts work together, quantum-safe and classical



Implementation and Architecture

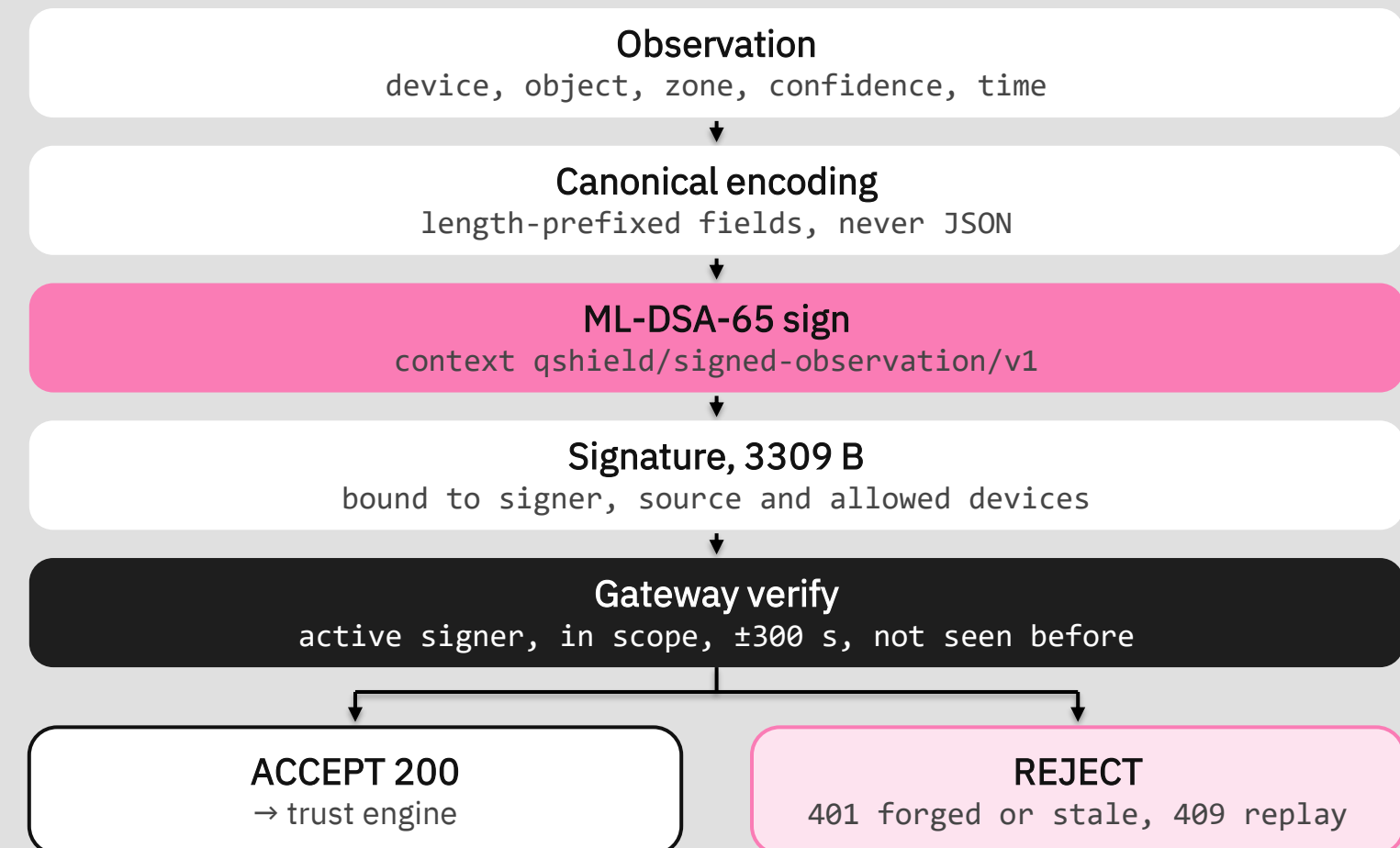
2 of 3: the post-quantum evidence path

ML-KEM-768 session (vision service ↔ gateway)

- 1 Gateway key pair, provisioned once
client pins the public key, 1184 B
- 2 Client encapsulates
→ shared secret + ciphertext, 1088 B
- 3 Init signed with ML-DSA-65; gateway verifies, decapsulates
→ shared secret, 32 B
- 4 HKDF-SHA256 on both sides
salt = both nonces, info = transcript
- 5 Key confirmation
a wrong pinned key: nothing is sent
- 6 AES-256-GCM session
per-direction keys, 64-bit counters, AAD

median, laptop: encaps 0.069 ms, decaps 0.091 ms, handshake 10.0 ms
Composed by us from standard primitives: not TLS, not externally reviewed.

ML-DSA-65 signed observation



Evidence chain: every decision SHA-256 linked and ML-DSA-65 signed; 26 entries
VERIFIED in the live demo.

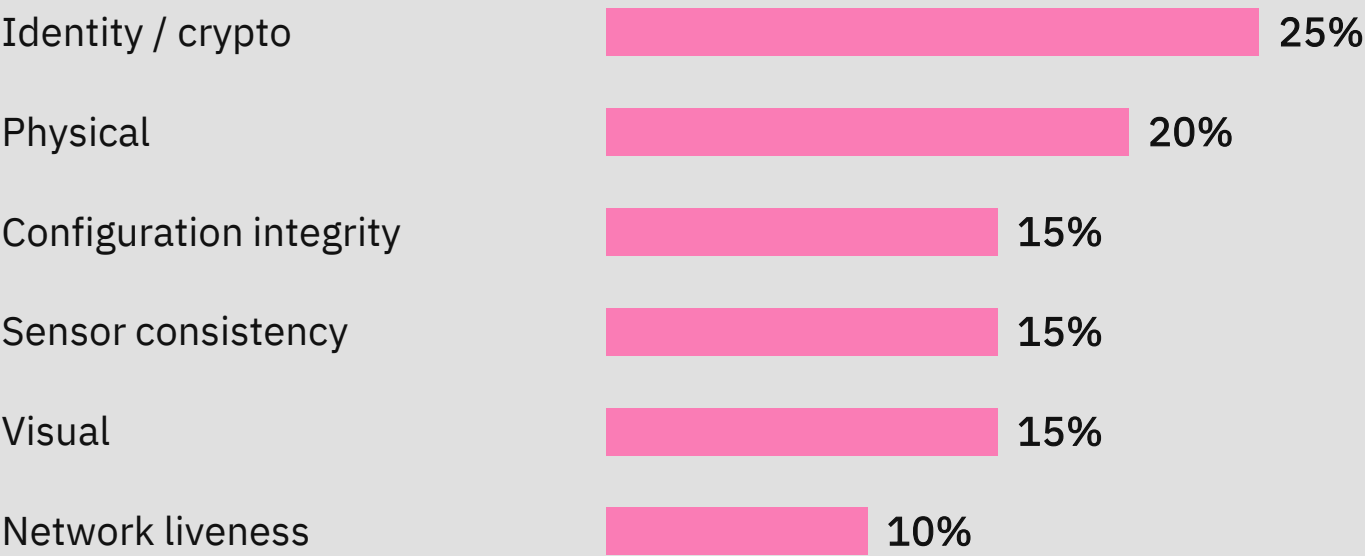




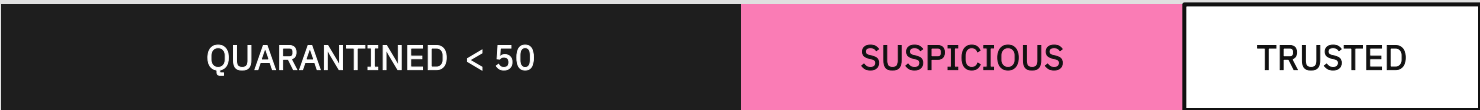
Implementation and Architecture

3 of 3: the trust engine and the camera as part of the security boundary

Six-factor trust



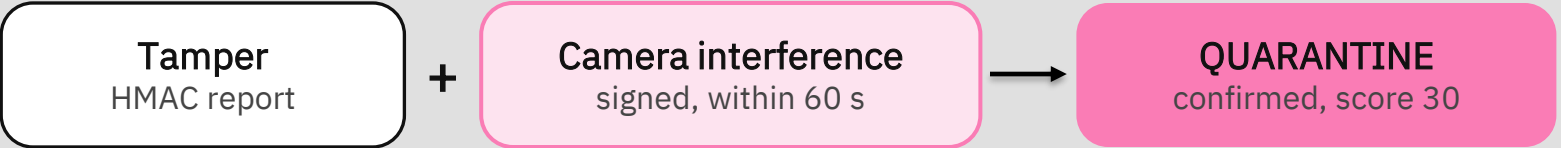
score = min(100 - Σ w_i · p_i - pressure, caps)



back to TRUSTED only at 85; pressure ≤ 25, so forged traffic alone never quarantines; only clean authenticated evidence earns recovery time

Camera checks (signed by the vision service)

State	How it is measured	Effect
Occluded	dark, flat or blinded for 2 s	HIGH
Moved	shifted, or similarity < 0.7	HIGH
Frozen	the same frame repeated	HIGH
Degraded	low light or blur	MEDIUM
Too close	box > 35% of frame, not a distance	LOW
Source lost	no frames	MEDIUM



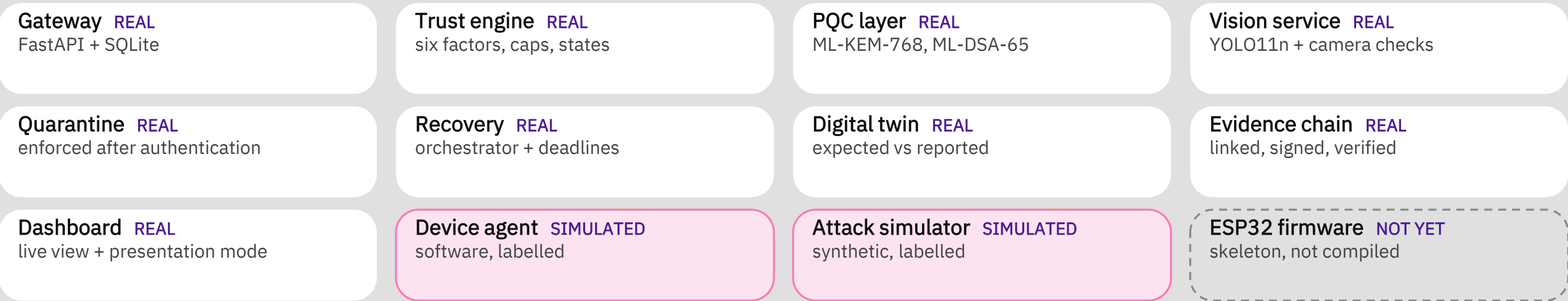
Camera alone: a penalty, never quarantine. A camera report and a detection from the same signer count once.



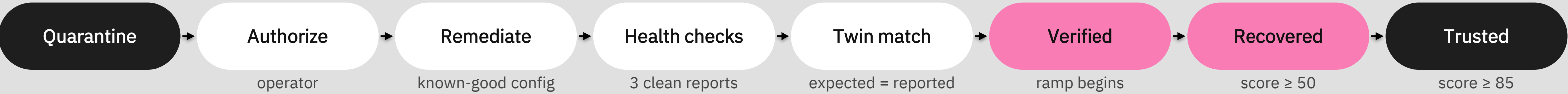


Minimal Prototype

What we have actually built: running software, tested end to end



Self-healing recovery



any failure → QUARANTINED: an authenticated fault, the 15-min verification deadline, the 2-h ramp deadline, revocation

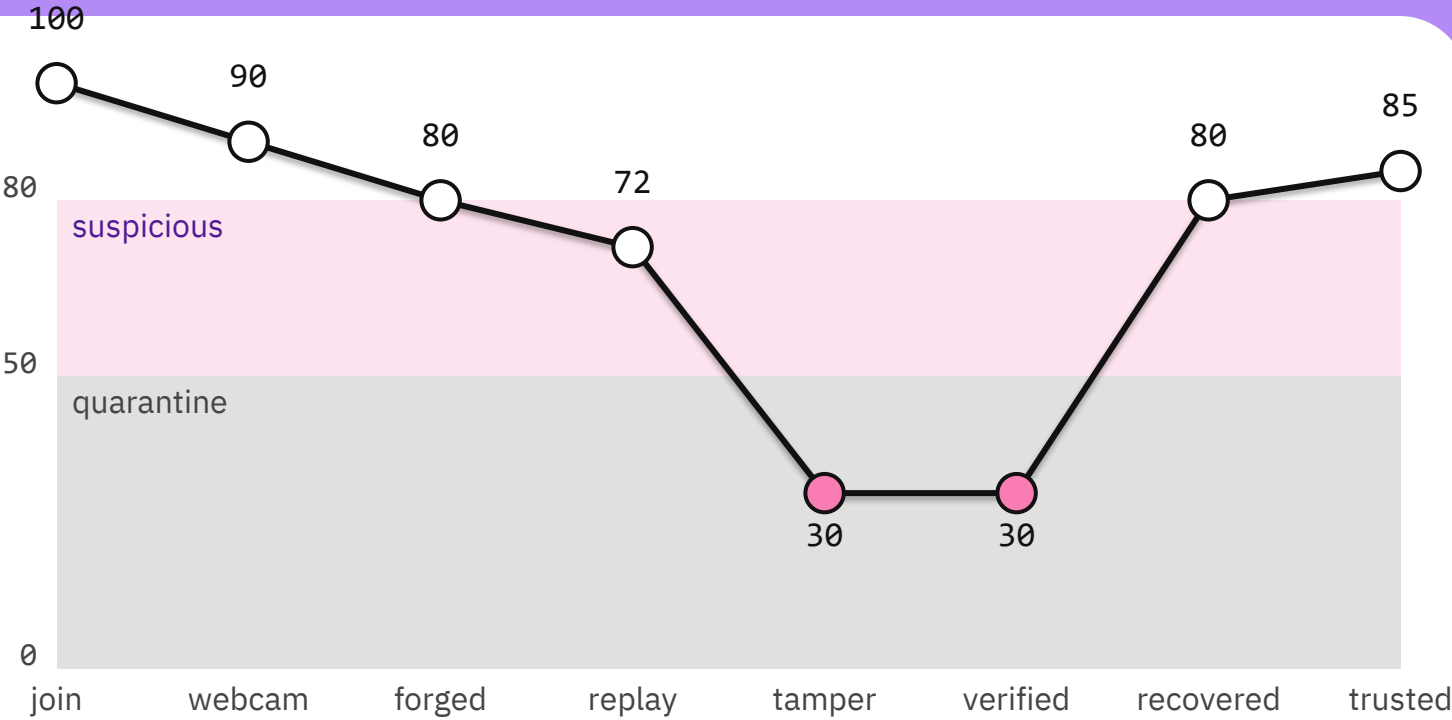
Run it: python scripts/demo_full.py --webcam --pace 2 --hold





Results / Validation

Validated on the real gateway with a real USB webcam; attacks are simulated and labelled



Live run, 2026-10-02; recovery shown in announced time-lapse

Forged observation

REJECTED 401

Replayed observation

REJECTED 409

Tamper + signed visual evidence

QUARANTINED, score 30

Forged 'all clear' in quarantine

401

Verified recovery

TRUSTED 85; chain VERIFIED

785

Python tests passing

40

JavaScript tests passing

70

NIST ACVP vectors agree

10.0 ms

full PQC handshake (median)

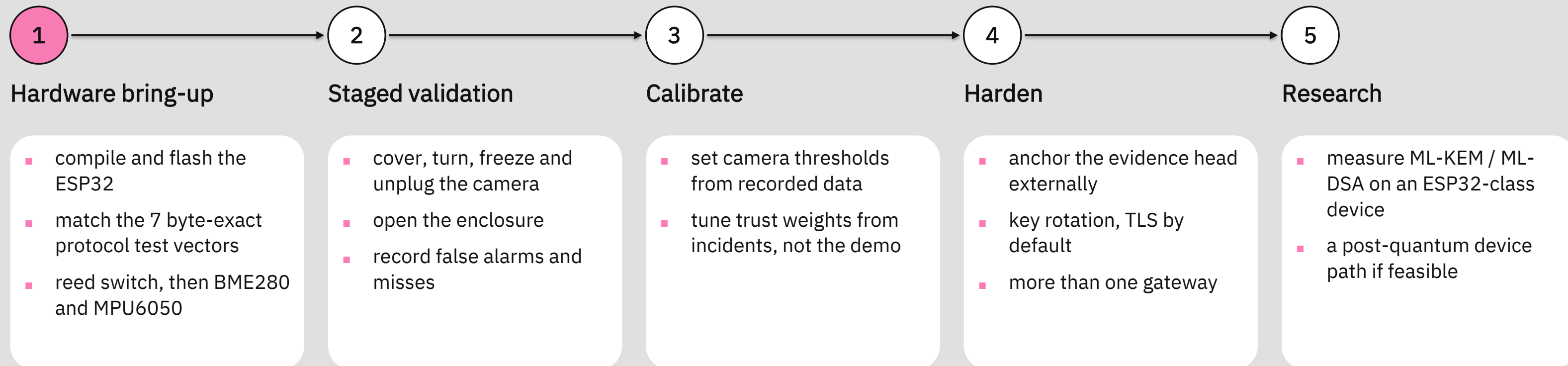
~5 fps

live USB webcam, no false camera alarm in 2 × 90 s

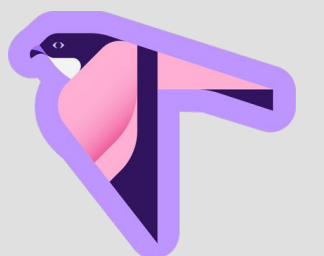


Roadmap

If selected for Round 2: from software prototype to a validated hardware deployment



Hardware is an adapter step: same protocol, same gateway, same trust engine (docs/hardware/).



References and Sources

Standards, research, libraries and our repository

Standards

- NIST FIPS 203, ML-KEM (2024) doi.org/10.6028/NIST.FIPS.203
- NIST FIPS 204, ML-DSA (2024) doi.org/10.6028/NIST.FIPS.204
- NIST SP 800-38D, GCM doi.org/10.6028/NIST.SP.800-38D
- NIST FIPS 180-4, SHA-256 doi.org/10.6028/NIST.FIPS.180-4
- RFC 5869 HKDF, RFC 2104 HMAC rfc-editor.org
- NIST ACVP test vectors github.com/usnistgov/ACVP-Server

Research

- P. W. Shor, Algorithms for quantum computation: discrete logarithms and factoring, FOCS 1994
- L. K. Grover, A fast quantum mechanical algorithm for database search, STOC 1996

Libraries and tools

- pqcrypto 1.0.0 (Apache-2.0): ML-KEM-768, ML-DSA-65
- cryptography (PyCA): AES-256-GCM, HKDF
- kyber-py, dilithium-py: test-only cross-checks
- Ultralytics YOLO11n (AGPL-3.0), OpenCV, NumPy
- FastAPI, Pydantic, SQLite

Our work

- github.com/Zynx095/Q-shield
- an original repository, not a fork; third-party libraries are dependencies under their licences
- docs/: specifications, test results and limitations

